

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE PILANI
K K BIRLA GOA CAMPUS

2025-26 Sem. I	Mid-Sem Regular	Open Book
Principles of Programming	CS F301	30%=60 marks; Pages=2;
Languages		Questions=3.18(i-v,1-10,1)
8 Oct 2025	90 min	0930-1100 hrs

Q1. We have seen many examples of C/C++ module independence. It boils down to the principle: “**many declarations, one unique definition**”, in which declarations allow different representations, and the definition fixes the imperative. Answer for each of the following, whether it can be different in different declarations in different modules or it has to be consistent in each module with the unique definition **with justification in exactly 1 line each:** [Marks **3x5=15**]

- (i) The size of the storage requirements of a data structure.
- (ii) The layout of its elements or members.
- (iii) The names and the number of member functions (methods) of a class.
- (iv) The signatures (prototypes) of member functions (methods) of a class.
- (v) The sequence of signatures (prototypes) and bodies of member functions (methods).

The answer needs to be in the following format

SrNo.	Independent?	Reason
(N)	Yes, since 1947	Quit India Movement, 1942
(∞)	No	It depends on the monsoon every year

Q2. Suppose in a C++ Module1 there is a global function with signature (in its body definition) as `void function(Class1 c)`. This function is neither a friend nor a member (method) of any class. In another Module2 a function prototype for `extern void function(Class2);` is declared and called as follows (the ellipses are for unseen but valid code, some of it detailing classes `Class1`, `Class2`):

```
extern void function(Class2);
```

```
...
```

```
... { // Some other function body
```

```
    Class2 c(...); // Instantiation
```

```
...
```

```
    function(c); // Call
```

```
}
```

Some additions are suggested for compiling and linking these two modules successfully as follows.

1. A body `void function(Class2 c) {...}` in Module1.
2. A body `void function(Class2 c) {...}` in Module2.
3. A body `void function(Class2 c) {...}` in any (and only one) one of Module1 and Module2.
4. A body `void function(Class2 c) {...}` each in each of Module1 and Module2.

5. An identical (except for whitespace) body `void function(Class2 c) {...}` each in each of Module1 and Module2.
6. `Class2` extending (derived from) `Class1`.
7. A prototype declaration `extern void function(Class1);` in Module1 at the top.
8. A prototype declaration `extern void function(Class1);` in Module2 at the top.
9. A prototype declaration `extern void function(Class1);` in either Module1 or Module2 but not both, at the top.
10. A prototype declaration `extern void function(Class1);` each in each of Module1 and Module2 at the top.

Indicate for each of the 10 additions, whether it is necessary, sufficient, or harmful for linking the two modules successfully. Necessary means that addition must be there, regardless of other additions. Sufficient means that addition makes the two modules linkable, no more changes needed. Harmful means that addition makes the modules incompatible in linking unless something is deleted. [Marks **3x10=30**]

The answer needs to be in the following format.

No.	Necessary	Sufficient	Harmful
11	✓		
12			✓

Consistency is important: something cannot be necessary yet harmful, for example. Overwriting, unclear marking, etc. will be interpreted as per the examiners' choices. Recheck requests on such clumsy answers will yield penalties. If you want to say "no" in any cell in the table, it must be a very white blank.

Q3. In your implementation of the last assignment (design of a list structures class hierarchy, in Data Abstraction and Object Orientation), did you make the storage structure (that actually contains the list elements in an aggregate) a base (super) class or a member (contain-&-delegate solution) of the classes `Queue`, `Stack`, `Deque`, `PriorityQ` etc.? Why? Which of the two design approaches is better for the two-way flexibility (making storage implementation variations and top level access classes variations independent of one another) sought? Justify. [Marks **2+5+8=15**]

S O L U T I O N

Answer1. This is based on the assignments and quizzes up till the Procedural to OO section.

SrNo.	Independent?	Reason
(i)	Yes	Sizes are dependent on procedure activation time declaration bindings.
(ii)	Yes	Again, procedure activation time declaration bindings.
(iii)	No	They depend on the unique definitions (bodies) of the member functions. There must be one-to-one exact matching correspondence.
(iv)	No	Same as (iii) above.
(v)	Should be	Sometimes some implementations may not allow this independence. The mechanism for matching method signatures with bodies and binding calls with declared signatures does not depend on the sequence but on matchings.
(v)	“Yes, independent	However, unless this implementation issue is mentioned, the correct answer is because what matters are unique signatures.”

Answer2. Again on C++ module independence, this is not about the procedural part but about the division of labour between the compiler and the linker/loader, plus OO type inference in C++. In the given scenario, nothing works unless there is a unique global body of `void function(Class2)` signature is provided in one of the modules. Curiously, if we change the top declaration in Module2 to `extern void function(Class1)`, i.e. matching the given body, then making `Class2` a subclass (derived) of `Class1` is sufficient.

No.	Necessary	Sufficient	Harmful	
1		✓		
2		✓		
3	✓	✓		
4			✓	
5			✓	
If void	we	do	not	remove
6	function	(Class2)	declaration	Irrelevant
7				✓
8				✓
9				✓
10				✓